PROMPT MASTER — ADAPTAÇÃO COMPLETA DE PROJETOS PARA CLOUDFLARE PAGES

1. OBJETIVO PRINCIPAL

Analise integralmente o projeto atual e adapte sua arquitetura, código-fonte, dependências e configurações para funcionar corretamente no Cloudflare Pages, utilizando o GitHub como repositório de versionamento e publicação automática.

O projeto já existe e possui funcionalidades implementadas.

REGRA FUNDAMENTAL: NÃO RECRIAR O PROJETO DO ZERO, NÃO ELIMINAR FUNCIONALIDADES EXISTENTES E NÃO ALTERAR RECURSOS QUE NÃO PRECISAM DE MODIFICAÇÃO.

Preserve integralmente:

- Funcionalidades existentes.
- Layout e identidade visual.
- Componentes e páginas.
- Responsividade.
- Banco de dados e integrações.
- Autenticação.
- APIs.
- Regras de negócio.
- Configurações existentes.
- Recursos implementados anteriormente.

Qualquer modificação deverá ser tecnicamente justificada.

2. AUDITORIA COMPLETA DO PROJETO

Antes de realizar qualquer alteração, examine todos os arquivos e diretórios.

Identifique:

- Framework utilizado.
- Linguagens de programação.

- Versões das dependências.
- Estrutura do frontend.
- Estrutura do backend.
- Sistema de rotas.
- APIs internas e externas.
- Banco de dados.
- Sistema de autenticação.
- Bibliotecas e módulos utilizados.
- Arquivos de configuração.
- Variáveis de ambiente.
- Dependências incompatíveis com Cloudflare Workers Runtime.

Identifique problemas que possam impedir a instalação, compilação, execução ou publicação.

Não considere o projeto compatível sem verificar sua estrutura real.

3. ARQUITETURA DE HOSPEDAGEM

Priorize a seguinte arquitetura, quando compatível:

Frontend:

- React.
- TypeScript.
- Vite.
- HTML5.
- CSS3.
- JavaScript moderno.

Hospedagem:

- Cloudflare Pages.

Backend:

- Cloudflare Pages Functions.
- APIs compatíveis com Cloudflare Workers Runtime.

Banco de dados, quando necessário:

- Cloudflare D1.
- Cloudflare KV.
- Cloudflare R2.
- Cloudflare Durable Objects.
- Bancos externos compatíveis.

Escolha somente os serviços necessários para cada aplicação.

Não adicionar dependências ou serviços desnecessários.

Se o projeto utilizar outra tecnologia, avalie sua compatibilidade antes de decidir pela migração.

4. COMPATIBILIDADE COM CLOUDFLARE PAGES

Verifique e corrija:

- Comandos de compilação.
- Configurações do Vite.
- Configurações do TypeScript.
- Caminhos de arquivos.
- Imports e exports.
- Dependências.
- Rotas internas.
- Navegação entre páginas.
- Carregamento de imagens.
- Fontes e recursos estáticos.
- Requisições HTTP.
- Tratamento de erros.
- Configurações de produção.

Configure a compilação para gerar corretamente a pasta de saída.

Para projetos React + Vite convencionais, utilizar como referência:

Build command: npm run build

Build output directory: dist

Production branch: main

Não aplicar essas configurações cegamente. Verificar primeiro os scripts e a estrutura real do projeto.

5. ADAPTAÇÃO DO BACKEND

Caso exista backend Node.js, Express ou outro servidor tradicional:

- Identificar todas as funcionalidades.
- Verificar dependências de APIs nativas do Node.js.
- Identificar incompatibilidades com Workers Runtime.
- Adaptar endpoints para Pages Functions quando possível.

- Preservar as regras de negócio.
- Preservar validações.
- Preservar autenticação e autorização.
- Preservar integrações externas.

Não tentar executar um servidor Express convencional diretamente como se o Cloudflare Pages fosse um servidor Node.js permanente.

Se alguma funcionalidade não puder ser adaptada ao Pages Functions, apresentar uma alternativa tecnicamente compatível, como Cloudflare Workers ou backend externo.

Não remover funcionalidades para forçar compatibilidade.

6. ESTRUTURA DE DIRETÓRIOS

Quando adequada ao projeto, utilizar uma organização semelhante a:

project/ ├── public/ ├── src/ │ ├── components/ │ ├── pages/ │ ├── services/ │ ├── hooks/ │ ├── types/ │ └── utils/ ├── functions/ │ └── api/ ├── package.json ├── package-lock.json ├── index.html ├── vite.config.ts ├── tsconfig.json ├── wrangler.toml ├── .gitignore └── .dev.vars.example

IMPORTANTE:

A pasta functions/ deve permanecer na raiz do projeto, fora de src/ e dist/.

Não mover arquivos sem necessidade.

Preservar a organização existente quando ela já estiver correta.

7. SEGURANÇA

Realizar auditoria de segurança completa.

Proibir:

- Chaves privadas no frontend.
- Senhas administrativas no código público.
- Tokens secretos no GitHub.
- Credenciais dentro de arquivos distribuídos ao navegador.
- Exposição de variáveis de ambiente.
- Endpoints administrativos sem autorização.

Implementar, quando necessário:

- Variáveis secretas no ambiente Cloudflare.
- Validação de entradas no servidor.
- Autenticação segura.
- Autorização por função ou permissão.
- Proteção contra requisições abusivas.
- Validação de uploads.
- Tratamento seguro de erros.
- Proteção de informações sensíveis.

Não considerar variáveis VITE_* secretas, pois são incorporadas ao código disponibilizado ao navegador.

Não publicar arquivos .env reais.

Criar apenas exemplos sem credenciais reais, como .dev.vars.example.

8. RESPONSIVIDADE E INTERFACE

Preservar o layout original.

Não realizar reformulação visual sem autorização.

Verificar:

- Celulares.
- Tablets.
- Computadores.
- Diferentes resoluções.
- Orientação vertical e horizontal.
- Navegação por teclado.
- Acessibilidade.
- Carregamento de fontes.
- Componentes responsivos.

Corrigir somente problemas efetivamente identificados.

9. CONFIGURAÇÃO DO GITHUB

Preparar o projeto para sincronização com GitHub.

Verificar:

- Estrutura do repositório.
- Arquivos desnecessários.
- .gitignore.

- Histórico de versões.
- Arquivos secretos.
- Dependências.
- Configurações de compilação.

Preparar o projeto para que alterações enviadas à branch main possam iniciar automaticamente um novo deployment no Cloudflare Pages.

Não sobrescrever repositórios ou histórico existente sem autorização.

10. CONFIGURAÇÃO DO CLOUDFLARE PAGES

Preparar todas as configurações necessárias.

Identificar:

- Framework preset.
- Build command.
- Build output directory.
- Root directory.
- Production branch.
- Variáveis de ambiente.
- Configurações de Functions.
- Configurações de rotas.
- Necessidade de configurações adicionais.

Criar ou atualizar arquivos de configuração somente quando necessário.

Não inserir credenciais reais.

11. TESTES OBRIGATÓRIOS

Executar os testes possíveis:

1. Instalação das dependências.
2. Verificação de dependências incompatíveis.
3. Verificação TypeScript.
4. Compilação de produção.
5. Testes das rotas.
6. Testes de APIs.
7. Testes de autenticação.
8. Testes de autorização.
9. Testes de navegação.
10. Testes de carregamento dos recursos.

11. Testes de responsividade.
12. Testes de upload e download, quando existentes.
13. Testes das integrações externas.
14. Verificação de erros no console.
15. Verificação de exposição de segredos.

Não afirmar que um teste foi aprovado sem realmente executá-lo.

Quando não for possível testar algo no ambiente disponível, identificar claramente a limitação.

12. PRESERVAÇÃO E RECUPERAÇÃO

Antes de alterações importantes:

- Preservar a estrutura original.
- Identificar arquivos modificados.
- Evitar exclusões desnecessárias.
- Manter histórico de alterações.
- Não substituir configurações sem verificar seu conteúdo.
- Não remover funcionalidades existentes.

Preparar o projeto para permitir recuperação de versões anteriores através do GitHub.

13. DOCUMENTAÇÃO DE ENTREGA

Ao terminar, apresentar um relatório técnico detalhado contendo:

A. Diagnóstico inicial.

B. Tecnologias identificadas.

C. Incompatibilidades encontradas.

D. Problemas corrigidos.

E. Arquivos modificados.

F. Arquivos adicionados.

G. Arquivos removidos e justificativas.

H. Dependências instaladas, atualizadas ou removidas.

I. Configuração final do Cloudflare Pages.

J. Configuração necessária no GitHub.

K. Variáveis de ambiente necessárias.

L. Configuração de backend e banco de dados.

M. Resultados dos testes.

N. Funcionalidades preservadas.

O. Funcionalidades que precisam de serviços externos.

P. Instruções para publicação.

Q. Problemas pendentes.

14. CRITÉRIOS DE APROVAÇÃO

O projeto somente poderá ser considerado preparado para publicação quando:

- A estrutura estiver compatível.
- As dependências estiverem verificadas.
- A compilação de produção funcionar.
- Os recursos originais estiverem preservados.
- As APIs necessárias estiverem funcionando ou devidamente preparadas.
- Não houver exposição de credenciais.
- As rotas estiverem configuradas.
- Os testes possíveis estiverem documentados.
- As configurações do Cloudflare estiverem identificadas.

Diferenciar claramente:

1. Projeto adaptado.
2. Projeto compilado.
3. Projeto testado.
4. Projeto publicado.
5. Projeto validado em produção.

Esses estados não são equivalentes.

15. INSTRUÇÃO FINAL

Comece realizando a auditoria integral do projeto atual.

Apresente o diagnóstico antes de fazer alterações estruturais.

Em seguida, execute as correções necessárias, preservando todas as funcionalidades existentes.

Não simplifique o projeto para facilitar a publicação.

Não invente implementações.

Não declare sucesso sem evidências.

O resultado deverá ser um projeto organizado, seguro, documentado, compatível com Cloudflare Pages e preparado para sincronização com GitHub e publicação automatizada.

PRIORIDADE ABSOLUTA: PRESERVAR A INTEGRIDADE DO PROJETO ORIGINAL E GARANTIR QUE TODAS AS FUNCIONALIDADES CONTINUEM OPERACIONAIS APÓS A ADAPTAÇÃO.